CHARM-SYCL & IRIS: A Toolchain for Performance Portability on Extremely Heterogeneous Systems

Norihisa Fujita*, Beau Johnston†, Narasinga Rao Miniskar†, Ryohei Kobayashi*,
Mohammad Alaul Haque Monil†, Seyong Lee†, Jeffrey S. Vetter†, Taisuke Boku*
*Center for Computatinonal Sciences, University of Tsukuba, Tsukuba, Ibaraki, Japan
†Oak Ridge National Laboratory, Oak Ridge, Tennessee, USA

***Abstract*—Performance portability is a crucial problem for extremely heterogeneous systems as HPC systems become increasingly heterogeneous. We have many options for CPUs and accelerators, such as, but not exclusively, GPUs, but also non-Von Neumann architectures. In this paper, we present the CHARM-SYCL unified programming environment for multiple accelerator types as a performance portable programming environment. It uses the IRIS library developed at Oak Ridge National Laboratory (ORNL) as the backend accelerator runtime. IRIS has a high-performance scheduler to distribute tasks across accelerators. This design allows us to run an application from the same source on multiple systems with multiple configurations. We provide three types of portability with CHARM-SYCL: *Portable Workflow*, *Compiler & Runtime Portability*, and *Application and Performance Portability*. We implement a Monte Carlo simulation benchmark code on the CHARM-SYCL programming environment and demonstrate that our programming environment can accommodate extremely heterogeneous systems.**

***Index Terms*—Extreme Heterogeneous System, SYCL, XS-Bench, Portability, Toolchain, Task System**

I. Introduction

Performance portability across systems presents a significant challenge in High Performance Computing (HPC). System configurations in HPC have become increasingly heterogeneous, posing system-specific questions on: What is the CPU architecture? How many accelerators? What type of accelerators? How are the accelerators connected (via PCIe, via direct connection such as NVLink)? Are CPUs and GPUs cache coherent? In practice, the US Department of Energy (DOE) operates three flagship supercomputers in the US. However, these systems have different accelerators from different vendors: AMD [1], Intel [2], and NVIDIA [3].

Applications implemented with portability in mind give us the freedom to choose the optimal system for running them. We can design HPC systems with the best choice of technology without vendor-lock-ins. This is a true codesign between computer science and computational science. We consider that keeping performance portability of applications is key for future-proofing extreme heterogeneous systems. To solve this challenge, we believe that having performance portable toolchains and development environments are essential. We also believe that the diversity of system architectures for HPC systems will continue to increase in the future, including not only CPU and/or GPU-based systems but also non-Von Neumann architectures such as Field Programmable Gate Arrays (FPGAs) [4].

In this paper, we present our CHARM-SYCL programming environment for multiple accelerator types [5] as a performance portable toolchain for extremely heterogeneous systems. The research goal of CHARM-SYCL is to realize the CHARM (Cooperative Heterogeneous Acceleration with Reconfigurable Multidevices) concept in an application that accelerates computation on heterogeneous systems. However, this paper focuses on the aspect of its performance portable toolchain.

The contributions of this paper are as follows.

- We propose our CHARM-SYCL compiler environment as a performance portable toolchain.
- We show the details of performance portability and how it is implemented.
- We show the performance evaluation results using the Monte Carlo simulation benchmark code.
- We demonstrate portable kernel execution through the scheduler in the IRIS runtime system [6], [7].

II. Related Work

The SYCL standard [8] is an open standard. As such, there are many different SYCL implementations including Intel's oneAPI [9], Intel/LLVM [10] (open source version of oneAPI), ComputeCpp [11], AdaptiveCPP [12], triSYCL [13], and neoSYCL [14]. Most of these SYCL implementations are built on the LLVM infrastructure and generate kernels through LLVM Intermediate Representation (IR) or Standard Portable Intermediate Representation (SPIR-V). Thanks to the design, oneAPI and AdaptiveCPP support not only CPUs but also a variety of accelerators including NVIDIA GPUs and AMD GPUs. However, they do not have enough portability for extreme heterogeneous systems. Namely, they do not have a portable compiler or runtime workflow. The compiler workflow requires the user to specify the target accelerator architecture at compile time, which result in multiple device-specific binaries. These multiple application binaries result in a hindered runtime workflow that can only use a single accelerator backend associated with a vendor—fragmenting a HPC